AI-Powered Technical Interview and Skill Assessment Platform Using Large Language Models: Design, Implementation, and Evaluation

Dhruv Parmar
Department of Computer Engineering
Silver Oak University
Ahmedabad, India
parmardhruv747@gmail.com

Pratiksinh Chauhan
Department of Computer Engineering
Silver Oak University
Ahmedabad, India
pratiksinh4645@gmail.com

Dhwij Patel
Department of Computer Engineering
Silver Oak University
Ahmedabad, India
pateldhwij217@gmail.com

Dr. Monali Suthar
Department of Computer Engineering
Silver Oak University
Ahmedabad, India

***Abstract*—Technical interviews are central to software engineering recruitment, yet students preparing for placements have limited access to realistic, structured practice with immediate feedback. This paper presents the design and implementation of an AI-powered technical interview and skill assessment platform in which a large language model (LLM), Google Gemini, acts as the interviewer while the application retains control of the interview structure. Candidates configure a session by role, experience level, difficulty, technologies and topics; the backend generates one question at a time, evaluates each answer against a four-criterion rubric, and allows at most one adaptive follow-up per question through a deterministic state machine. Coding answers written in an embedded Monaco editor are reviewed by the model for correctness, algorithm, code quality and Big-O complexity. Every model output is requested as JSON and validated with Zod schemas, and a browser-based proctoring module enforces fullscreen mode, detects tab switches and blocks clipboard and developer shortcuts. The system is implemented with React, TypeScript, Node.js and Express. In a functional evaluation with the live model, 22 of 25 test cases passed, and a case-study session produced role-specific questions, code-specific feedback and a final report that accounted for an unanswered question. We also report the limitations observed, including dependence on the availability and quota of the LLM service, and outline a user study to measure agreement with human interviewers.**

***Index Terms*—Large language models, technical interviews, automated assessment, code evaluation, prompt engineering, online proctoring.**

I. Introduction

Technical interviews remain a decisive step in software engineering hiring and in campus placement drives. Decades of selection research show that structured interviews, in which every candidate receives comparable questions scored against explicit criteria, predict job performance better than unstructured conversations [1], [2]. For students, however, structured practice is scarce: mock interviews with experienced engineers are costly and difficult to schedule, coding websites judge only whether submitted code passes test cases, and general-purpose chatbots neither follow an interview structure nor return consistent scores.

Instruction-tuned LLMs such as GPT-4, PaLM and Gemini can now generate domain-specific questions, reason about code and hold multi-turn conversations [3]–[5]. Studies of LLMs as evaluators report agreement with human preferences comparable to agreement between humans [6]. At the same time, LLM outputs can be fluent but unsupported, inconsistent between runs, or malformed [7], so an assessment tool cannot simply forward model output to the user.

This paper describes a platform that combines these capabilities with an application-controlled interview workflow. Its contributions are:

1. an interview workflow in which the application, not the model, owns progression: questions are generated lazily with topic rotation, and follow-ups are bounded by an explicit state machine;
2. a schema-validated prompting layer behind a provider-independent interface, which turns every model call into typed data and allows a mock provider for development;
3. joint assessment of conceptual and coding answers in one session, with model-based complexity analysis, a holistic final report and a deterministic fallback;
4. a browser-based proctoring module; and
5. a functional evaluation with the live model, together with an account of the limitations and defects it revealed.

Section II reviews related work, Section III presents the design, Section IV the evaluation, Section V the limitations and future work, and Section VI concludes.

II. Related Work

A. Automated Interview Assessment

MACH used a virtual agent that analysed facial expressions, speech and prosody to coach users in mock job interviews, and

participants who practised with it improved in subsequent interviews rated by career counsellors [8]. Naim et al. predicted human ratings of recorded mock interviews from lexical, prosodic and facial features [9], and HireNet applied hierarchical attention to asynchronous video interviews [10]. Hickman et al. examined automated video interview personality assessments and reported mixed evidence of reliability and validity, including limited generalizability to new samples [11]. Raghavan et al. found that vendors of algorithmic pre-employment assessments disclosed little about validation and bias mitigation [12]. These systems target behavioural cues in hiring decisions; our platform targets technical content in a practice setting and states its scoring criteria explicitly in the prompts.

B. LLMs for Code Understanding

Codex solved 28.8% of the 164 HumanEval problems with a single sample [13], and later models such as AlphaCode, StarCoder and Code Llama improved code generation further [14]–[16]; pre-training on code and comments also improves code representation [17]. We use a general-purpose model to review submitted code and estimate its complexity without executing it; the accuracy of such estimates is not assumed and is part of the planned evaluation.

C. Reliability of LLM Evaluators

Instruction tuning with human feedback makes models follow task instructions and personas [18], and asking for intermediate reasoning improves complex reasoning [19]. LLM judges, however, show position, verbosity and self-enhancement biases [6], and generated text can contain unsupported content [7]. Guardrail toolkits [20] and automated feedback loops [21] have been proposed to constrain outputs. Our platform constrains the form of the output with schemas; constraining its correctness remains future work.

D. LLMs in Education

Kasneci et al. discuss the opportunities of LLMs for personalised learning and feedback together with risks of bias and over-reliance [22], and a systematic review of automatic question generation found that most earlier systems were template-based and difficult to evaluate [23]. Our platform applies LLM question generation and feedback to interview practice.

III. System Design

A. Architecture

The platform is a three-tier web application (Fig. 1, Table I). The React single-page application provides the setup form, the interview workspace with chat, code and voice modes, the results view, history and a practice sandbox. It calls an Express REST API with a JSON Web Token. Each request passes through Helmet security headers, CORS restricted to the front-end origin, a 10 MB body limit, a rate limiter of 100 requests per 15 minutes per client, token authentication and Zod validation of request bodies. Services contain the business logic and depend on two abstractions: an `AIService` interface with six operations (question generation, follow-up generation, answer evaluation, coding evaluation, result generation and practice generation), implemented by a Gemini provider and a mock provider, and a data facade that uses Prisma with PostgreSQL when a database URL is configured and an in-memory store otherwise. The Gemini API key is read only on the server.

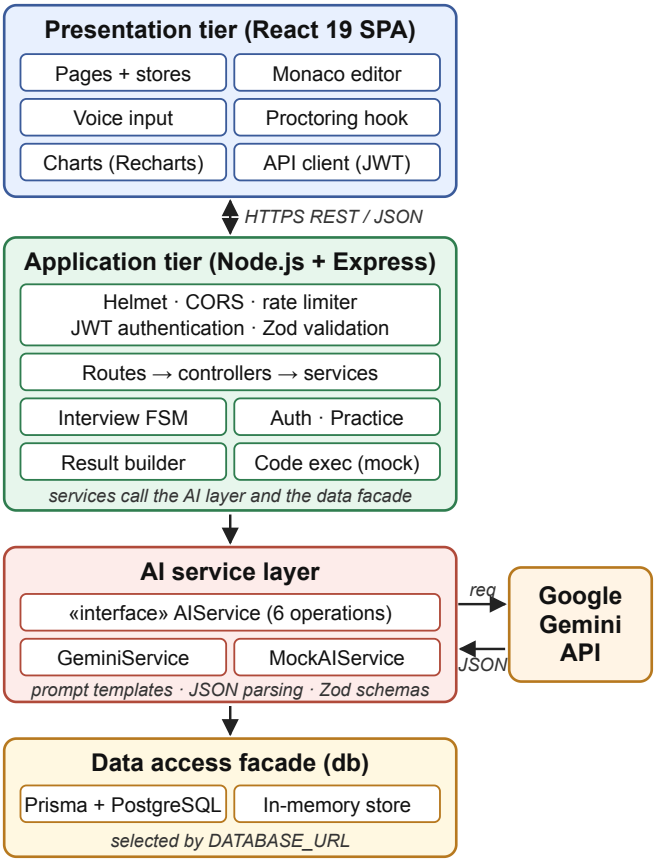


Fig. 1. System architecture. Services own the interview state; the LLM is reached only through the AI service layer, and every response is parsed and validated before use.

TABLE I
Implementation Stack

Layer	Technology
Frontend	React 19, TypeScript, Vite, Tailwind CSS, Zustand
Editor and charts	Monaco Editor (6 languages), Recharts
Browser APIs	Web Speech, Fullscreen, Page Visibility
Backend	Node.js, Express 4, TypeScript (ES modules)
LLM	Google Gemini API, model gemini-3.5-flash
Validation	Zod schemas for requests and model output
Security	bcrypt, JWT, Helmet, CORS, rate limiting
Persistence	Prisma ORM with PostgreSQL; in-memory store

B. Interview Workflow and State Machine

A candidate chooses one of ten roles, five experience levels, four difficulty levels, seven interview types, technologies, optional topics, a duration and 5, 10, 15 or 20 questions (N). The session is stored as *pending*. When it starts, the service marks

it *active* and generates only the current question, choosing its topic by rotation over the topic list T:

$$topic(q) = T[\, q \bmod |T| \,] \tag{1}$$

where q is the zero-based question index. Each later question is generated when the session reaches it, so model calls match the questions actually asked.

Progression is modelled by the state (q, k), where $k \in \{0, 1\}$ is the number of answers already stored for question q (Fig. 2). After an answer is evaluated, let E be the evaluator's follow-up flag. A follow-up is asked only if

$$F = E \wedge (k = 0) \wedge (q < N - 1) \tag{2}$$

If F holds, the model is asked for a follow-up question, which it may decline (a default question about trade-offs is used if the call fails); the follow-up is stored with the answer and the state becomes $(q, 1)$. Otherwise the state becomes $(q + 1, 0)$, and the session completes when $q + 1 = N$. Hence every question receives at most one follow-up, the last question receives none, and a session ends after at most $2N - 1$ answer submissions. The candidate can also complete the session early, and it is completed automatically when the timer expires or on the third tab-switch violation.

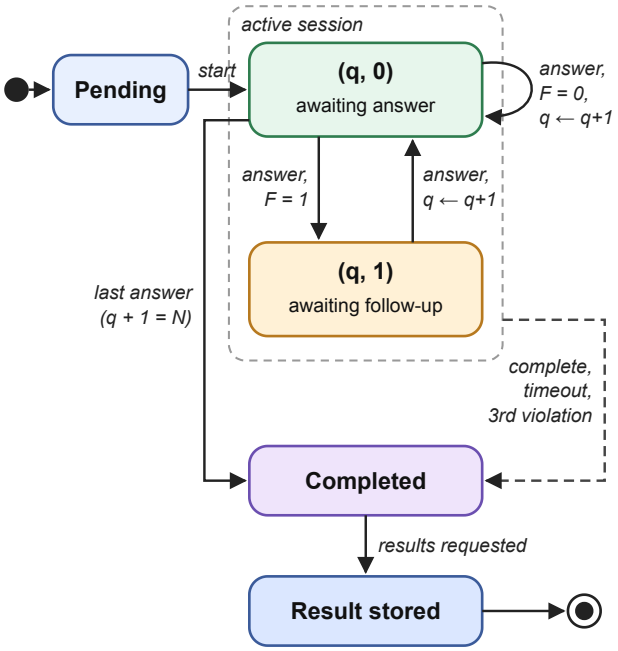


Fig. 2. Interview state machine. q is the question index, N the number of questions and F the follow-up condition of (2).

C. Prompt Design and Output Validation

Every model call uses a fixed system instruction that defines the interviewer persona: a senior, professional interviewer who does not help the candidate or reveal answers, asks concise questions, tests understanding rather than memorisation, asks for examples and trade-offs, keeps a neutral tone and evaluates only job-relevant skills. Each task prompt lists the exact JSON fields required, and the helper appends an instruction to return JSON only. The response is stripped of Markdown code fences, parsed, and validated with a Zod schema. Schemas for questions, answer evaluations and follow-ups supply defaults for missing optional fields, while the coding-evaluation and result schemas require every field. A response that is not valid JSON or fails a required field raises an error; the current implementation does not retry the call. Table II lists the fields returned for each answer type.

TABLE II
EVALUATION FIELDS RETURNED BY THE MODEL

Aspect	Conceptual answer	Coding answer
Overall (0–100)	score	score
Correctness	technicalAccuracy	correctness
Depth / quality	depth	codeQuality
Reasoning	problemSolving	algorithm
Communication	communication	fixed at 80
Complexity	—	time, space (Big-O)
Narrative	feedback, strengths, weaknesses	feedback, strengths, improvements
Follow-up flag	shouldAskFollowUp	not used

D. Coding Evaluation

Coding questions open in a Monaco editor with starter code from the model and support JavaScript, TypeScript, Python, Java, C++ and Go. Code execution is behind a provider interface whose current implementation returns simulated output, so submissions are assessed by the model alone, which reports correctness, algorithm and code quality ratings and Big-O time and space complexity. The ratings are mapped to the conceptual fields (correctness to accuracy, code quality to depth, algorithm to problem solving) so that both answer types can be aggregated.

E. Result Generation

On completion, the service sends the full history of questions, answers, scores and feedback to the model and requests an overall score, five skill scores (technical knowledge, problem solving, coding, communication and accuracy), strengths, weaknesses, recommendations and a summary. The result is stored once and reused. If the call fails, scores are computed from the n stored answers, for example

$$S = round(\,(1/n)\,\Sigma_i\, s_i\,), \quad C = round(\,(1/n)\,\Sigma_i\, c_i\,) \tag{3}$$

where S is the overall score, C the coding score, s_i the score of answer i, and c_i equals s_i for code answers and the technical accuracy otherwise; the remaining skills average their corresponding fields, and all scores default to 60 when no answer exists.

F. Proctoring

Before the interview starts, the candidate accepts the proctoring rules. A React hook then requests fullscreen mode and requests it again if the candidate leaves; listens for page-visibility changes and shows a warning overlay for each tab switch, completing the interview on the third; cancels copy,

cut and context-menu events and the shortcuts Ctrl/Cmd+C, V, X, P, S, U and I, F12 and Ctrl/Cmd+Shift+I, J and C; neutralises the Back button; and asks for confirmation before the page is closed. All rules are removed when the session ends.

G. Security, Persistence and Practice

Passwords are hashed with bcrypt using 10 salt rounds [24], and tokens are signed JWTs valid for seven days [25], sent as an HTTP-only cookie or a Bearer header. The Prisma schema defines seven tables (users, interviews, questions, answers, results, skill scores and practice questions) with cascading deletes. The practice sandbox generates three questions with hints and a model solution for a chosen category and difficulty and evaluates answers with the interview rubric.

IV. Evaluation

A. Setup

The platform was tested in October 2026 with the live Gemini model (`gemini-3.5-flash`), the in-memory store, the Vite development server and Node.js. REST endpoints were tested with direct HTTP requests, user flows with a scripted Chrome browser, and proctoring with keyboard, clipboard and context-menu events and real tab switches in the locked workspace. The aim was to verify functional behaviour; agreement with human raters was not measured.

B. Functional Testing

Twenty-five test cases were executed (Table III). All nine authentication and validation cases behaved correctly, although duplicate registration and wrong-password login returned HTTP 500 instead of 409 and 401. Topic rotation followed (1): the first three questions covered JavaScript, React and Databases as configured. All six blocked actions were cancelled while ordinary typing was not, the overlay reported "Warning 1 of 3", and the third switch completed the session. The follow-up path for conceptual answers could not be exercised because the daily request quota of the API key was exhausted.

TABLE III
Functional Test Results

Category	Cases	Pass	Fail	Not run
Authentication and validation	9	9*	0	0
Interview workflow and AI evaluation	9	8	0	1
Code execution (simulated)	1	1	0	0
Proctoring	3	3	0	0
Security headers and rate limit	1	1	0	0
Error handling and analytics	2	0	2	0
Total	**25**	**22**	**2**	**1**

* Two cases rejected the input correctly but returned HTTP 500 instead of 409 or 401.

The two failures were (i) temporary model errors (HTTP 503) and quota errors (HTTP 429), which surfaced as a generic message with no retry, and (ii) dashboard statistics, which showed 0/100 for completed sessions because the average is read from a field that result generation does not update.

C. Case Study

A five-question session was configured for a junior Full Stack Developer at medium difficulty with the topics JavaScript, React, Databases, APIs and Algorithms. Table IV shows the evaluations returned for the questions reached before the session was completed early.

TABLE IV
Model Evaluation in the Case-Study Session

Q	Topic, type	Score	Acc.	Qual.	Alg.	Big-O (t / s)
1	JavaScript, coding	100	100	100	100	O(1) / O(1)
2	React, coding	92	95	95	90	O(1) / O(1)
3	Databases, conceptual	not answered				

Question 1 asked for a function that combines three asynchronous calls with concurrency and fallbacks; question 2 asked for a React hook that avoids stale responses. The feedback referred to specific elements of the submitted code, and for question 2 it suggested cancelling stale requests with an `AbortController`, a valid improvement. The final report gave Overall 64, Technical Knowledge 65, Problem Solving 65, Coding 68, Communication 85 and Accuracy 64, and its weaknesses named the unanswered SQL question. The arithmetic fallback of (3) would have given 96 for the two answered questions, which shows the value of evaluating the whole session. The fallback itself was verified on a separate session with one coding answer (s = 98, accuracy 100, algorithm 100, communication 80), which produced 98, 100, 100, 98, 80 and 100, as (3) predicts.

D. Discussion

The results show that the workflow, validation, proctoring and fallback mechanisms behave as designed with a real model. They do not show that the scores are valid: the model's ratings were not compared with human interviewers, the same answer may be scored differently in different runs [6], and complexity estimates are produced without running the code. The number of sessions was also limited by the API quota.

V. Limitations and Future Work

First, score validity is unmeasured. We plan a study in which candidates complete sessions that are scored independently by the platform and by two faculty interviewers using the same rubric, reporting correlation and intraclass correlation together with a candidate experience survey. Second, the platform depends on an external API: retries with back-off, clear quota messages and a fallback model are needed. Third, candidate answers are inserted directly into evaluation prompts, so an answer could contain instructions aimed at the evaluator; prompt injection and jailbreaking are documented risks for LLM-integrated applications [26], [27], and candidate text should be delimited and screened. Fourth, code is not executed; a sandbox such as Judge0 with hidden tests should comple-

ment the model review. Fifth, browser-based proctoring can be bypassed with a second device, and violations are counted only in the browser rather than recorded on the server. Finally, testing revealed implementation defects that we will correct: duplicate generation of the first question when the start request is sent twice in development mode, a timer fixed at 45 minutes regardless of the configured duration, dashboard statistics not linked to stored results, and authentication errors returned as HTTP 500. Further directions include spoken interviews, difficulty calibration with item response theory, and resume-based question generation.

VI. Conclusion

We presented an AI-powered technical interview and skill assessment platform in which an LLM generates and evaluates questions while the application controls progression through a bounded state machine and validates every model output against a schema. The platform assesses conceptual and coding answers in one proctored session and produces a holistic report with a deterministic fallback. A functional evaluation with the live model confirmed the intended behaviour in 22 of 25 cases and exposed concrete weaknesses in error handling and analytics. Measuring agreement with human interviewers is the next step before such scores can be relied upon beyond practice.

Acknowledgment

This work was carried out as minor project CSE_M2_35 under the guidance of Dr. Monali Suthar in the Department of Computer Engineering, Silver Oak University, Ahmedabad.

References

[1] F. L. Schmidt and J. E. Hunter, "The validity and utility of selection methods in personnel psychology: Practical and theoretical implications of 85 years of research findings," *Psychol. Bull.*, vol. 124, no. 2, pp. 262–274, 1998.

[2] P. R. Sackett, C. Zhang, C. M. Berry, and F. Lievens, "Revisiting meta-analytic estimates of validity in personnel selection: Addressing systematic overcorrection for restriction of range," *J. Appl. Psychol.*, vol. 107, no. 11, pp. 2040–2068, 2022.

[3] OpenAI, "GPT-4 technical report," arXiv:2303.08774, 2023.

[4] A. Chowdhery *et al.*, "PaLM: Scaling language modeling with pathways," *J. Mach. Learn. Res.*, vol. 24, no. 240, pp. 1–113, 2023.

[5] Gemini Team, Google, "Gemini: A family of highly capable multimodal models," arXiv:2312.11805, 2023.

[6] L. Zheng *et al.*, "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), Datasets and Benchmarks Track*, vol. 36, 2023.

[7] Z. Ji *et al.*, "Survey of hallucination in natural language generation," *ACM Comput. Surv.*, vol. 55, no. 12, pp. 1–38, 2023.

[8] M. E. Hoque, M. Courgeon, J.-C. Martin, B. Mutlu, and R. W. Picard, "MACH: My automated conversation coacH," in *Proc. ACM Int. Joint Conf. Pervasive Ubiquitous Comput. (UbiComp)*, 2013, pp. 697–706.

[9] I. Naim, M. I. Tanveer, D. Gildea, and M. E. Hoque, "Automated analysis and prediction of job interview performance," *IEEE Trans. Affect. Comput.*, vol. 9, no. 2, pp. 191–204, 2018.

[10] L. Hemamou, G. Felhi, V. Vandenbussche, J.-C. Martin, and C. Clavel, "HireNet: A hierarchical attention model for the automatic analysis of asynchronous video job interviews," in *Proc. AAAI Conf. Artif. Intell.*, vol. 33, 2019, pp. 573–581.

[11] L. Hickman, N. Bosch, V. Ng, R. Saef, L. Tay, and S. E. Woo, "Automated video interview personality assessments: Reliability, validity, and generalizability investigations," *J. Appl. Psychol.*, vol. 107, no. 8, pp. 1323–1351, 2022.

[12] M. Raghavan, S. Barocas, J. Kleinberg, and K. Levy, "Mitigating bias in algorithmic hiring: Evaluating claims and practices," in *Proc. Conf. Fairness, Accountability, and Transparency (FAT*)*, 2020, pp. 469–481.

[13] M. Chen *et al.*, "Evaluating large language models trained on code," arXiv:2107.03374, 2021.

[14] Y. Li *et al.*, "Competition-level code generation with AlphaCode," *Science*, vol. 378, no. 6624, pp. 1092–1097, 2022.

[15] R. Li *et al.*, "StarCoder: May the source be with you!" *Trans. Mach. Learn. Res.*, 2023.

[16] B. Rozière *et al.*, "Code Llama: Open foundation models for code," arXiv:2308.12950, 2023.

[17] D. Guo, S. Lu, N. Duan, Y. Wang, M. Zhou, and J. Yin, "UniXcoder: Unified cross-modal pre-training for code representation," in *Proc. 60th Annu. Meeting Assoc. Comput. Linguistics (ACL)*, 2022, pp. 7212–7225.

[18] L. Ouyang *et al.*, "Training language models to follow instructions with human feedback," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 35, 2022, pp. 27730–27744.

[19] J. Wei *et al.*, "Chain-of-thought prompting elicits reasoning in large language models," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 35, 2022, pp. 24824–24837.

[20] T. Rebedea, R. Dinu, M. N. Sreedhar, C. Parisien, and J. Cohen, "NeMo Guardrails: A toolkit for controllable and safe LLM applications with programmable rails," in *Proc. Conf. Empirical Methods Natural Lang. Process.: Syst. Demonstrations (EMNLP)*, 2023.

[21] B. Peng *et al.*, "Check your facts and try again: Improving large language models with external knowledge and automated feedback," arXiv:2302.12813, 2023.

[22] E. Kasneci *et al.*, "ChatGPT for good? On opportunities and challenges of large language models for education," *Learn. Individ. Differ.*, vol. 103, Art. no. 102274, 2023.

[23] G. Kurdi, J. Leo, B. Parsia, U. Sattler, and S. Al-Emari, "A systematic review of automatic question generation for educational purposes," *Int. J. Artif. Intell. Educ.*, vol. 30, pp. 121–204, 2020.

[24] N. Provos and D. Mazières, "A future-adaptable password scheme," in *Proc. USENIX Annu. Tech. Conf., FREENIX Track*, 1999.

[25] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," IETF RFC 7519, May 2015.

[26] Y. Liu *et al.*, "Jailbreaking ChatGPT via prompt engineering: An empirical study," arXiv:2305.13860, 2023.

[27] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. 16th ACM Workshop Artif. Intell. Security (AISec)*, 2023, pp. 79–90.